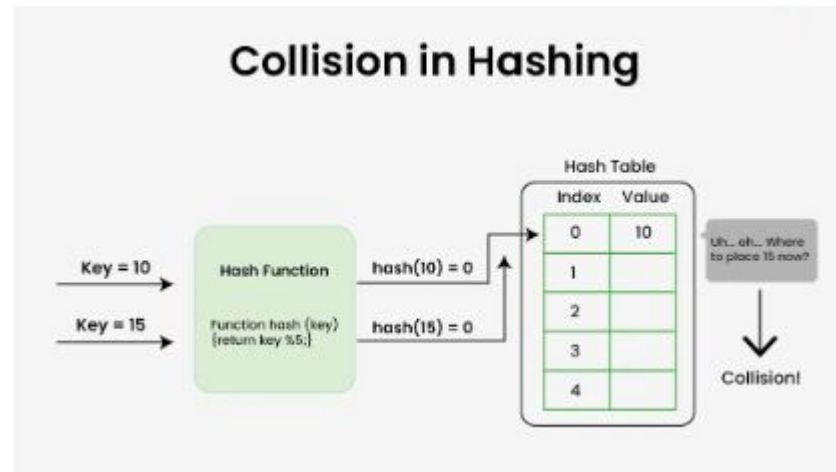
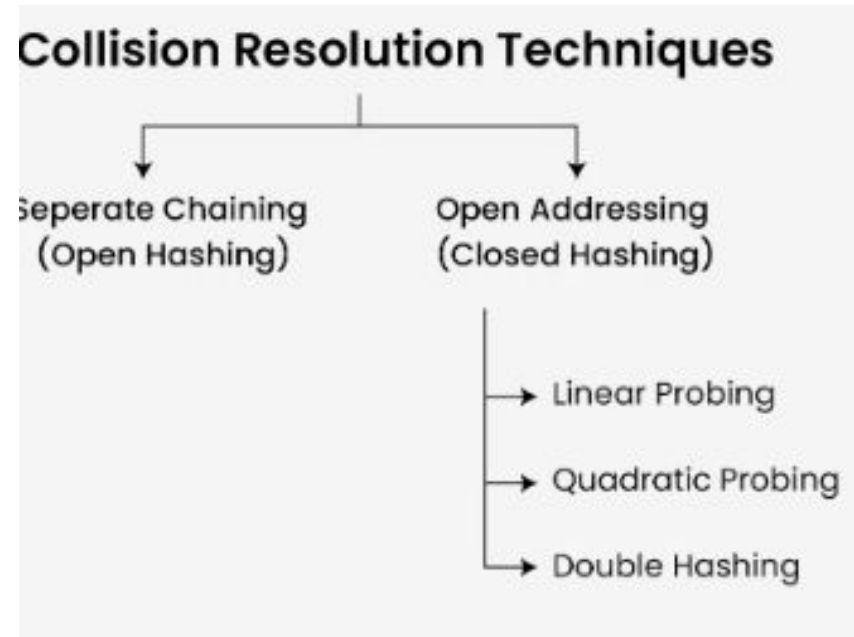


collision in hashing

- In hashing, a collision occurs when two different input values produce the same hash value, meaning they are mapped to the same location in a hash table.
- This happens because the hash function, designed to compress a large set of input data into a smaller range, may not always be able to create unique hash values for every distinct input.



Collision Resolution Techniques



Collision Resolution Techniques

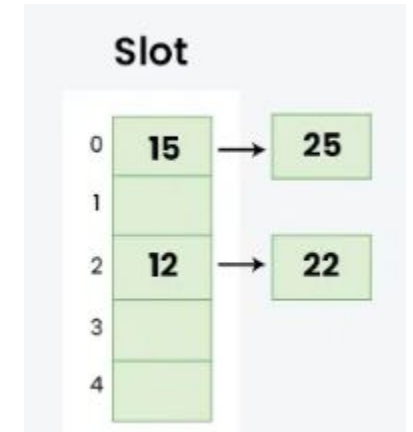
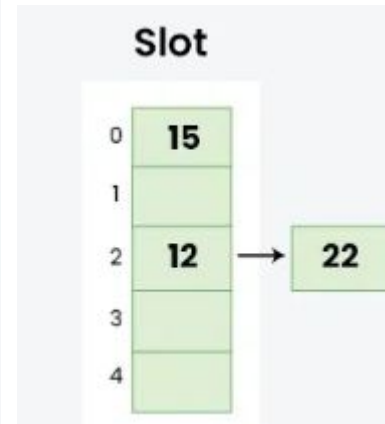
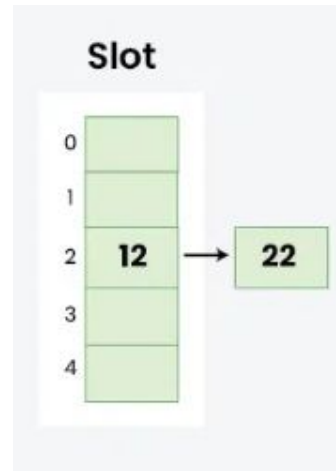
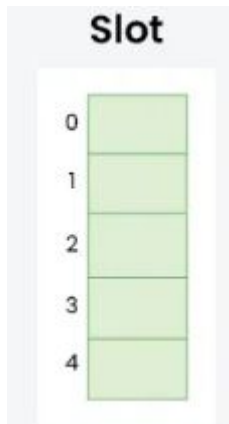
- **1) Separate Chaining**

- The idea behind Separate Chaining is to make each cell of the hash table point to a linked list of records that have the same hash function value. Chaining is simple but requires additional memory outside the table.

Separate Chaining

Hash function = $\text{key} \% 5$,

Elements = 12, 15, 22, 25 and 37.

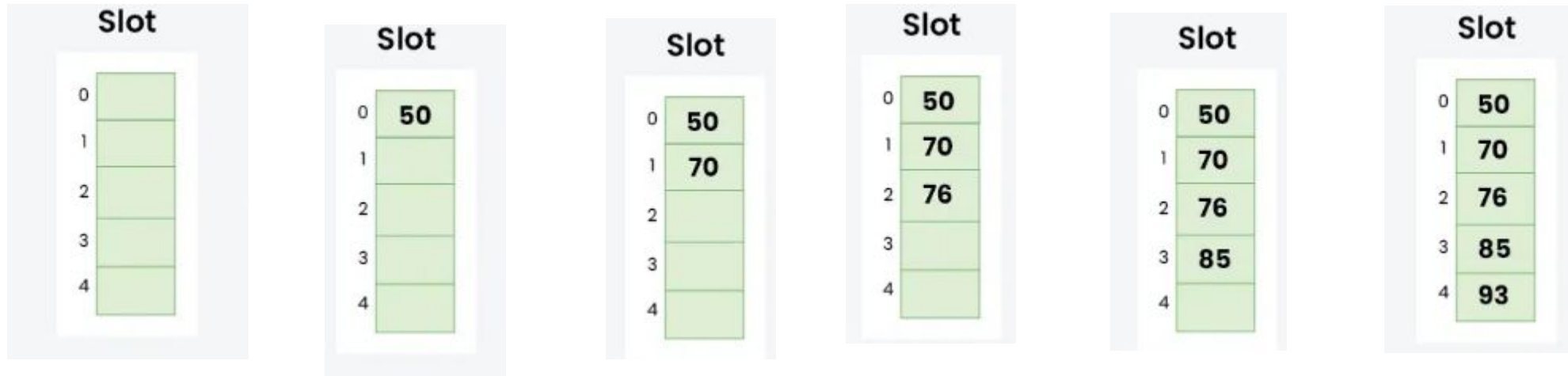


Open Addressing

- In open addressing, all elements are stored in the hash table itself. Each table entry contains either a record or NIL. When searching for an element, we examine the table slots one by one until the desired element is found or it is clear that the element is not in the table.

Linear Probing

- Let us consider a simple hash function as “key mod 5” and a sequence of keys that are to be inserted are 50, 70, 76, 85, 93.



Quadratic Probing

- Quadratic probing is an open addressing scheme in computer programming for resolving hash collisions in hash tables. Quadratic probing operates by taking the original hash index and adding successive values of an arbitrary quadratic polynomial until an open slot is found.

Quadratic Probing

- Insert = 22, 30, and 50

Slot

0	
1	
2	
3	
4	
5	
6	

Step 01

Empty hash table with range of hash values from 0 to 6 according to the hash function provided.

Slot

0	
1	22
2	
3	
4	
5	
6	

Step 02

The first key to be inserted is 22 which is mapped to slot 1 ($22\%7=1$)

Slot

0	
1	22
2	30
3	
4	
5	
6	

Step 03

The next key is 30 which is mapped to slot 2 ($30\%7=2$)

Slot

0	
1	22
2	30
3	
4	
5	50
6	

← $1+0$

← $1+1^2$

← $1+2^2$

Step 04

The next key is 50 which is mapped to slot 1 ($50\%7=1$) but slot 1 is already occupied. So, we will search slot $1+1^2$, i.e. $1+1 = 2$. Again slot 2 is occupied, so we will search cell $1+2^2$, i.e. $1+4 = 5$,

Double Hashing

- Double hashing is a collision resolving technique in Open Addressed Hash tables. Double hashing make use of two hash function,
- The first hash function is **$h_1(k)$** which takes the key and gives out a location on the hash table. But if the new location is not occupied or empty then we can easily place our key.
- But in case the location is occupied (collision) we will use secondary hash-function **$h_2(k)$** in combination with the first hash-function **$h_1(k)$** to find the new location on the hash table.

This combination of hash functions is of the form

$$h(k, i) = (h_1(k) + i * h_2(k)) \% n$$

Double Hashing

Double Hashing (Open Addressing)

Slot

0	
1	
2	
3	
4	
5	
6	

Step 01

Empty hash table with range of hash values from 0 to 6 according to the hash function provided.

Slot

0	
1	
2	
3	
4	
5	
6	27

Step 02

The first key to be inserted is 27 which is mapped to slot 6 ($22\%7=6$).

Slot

0	
1	43
2	
3	
4	
5	
6	27

Step 03

The next key is 43 which is mapped to slot 1 ($43\%7=1$).

Slot

0	
1	43
2	692
3	
4	
5	
6	27

Step 04

The next key is 692 which is mapped to slot 6 ($692\%7=6$), but location 6 is already occupied. Using double hashing,
$$h_{new} = [h_1(692) + i * (h_2(692))] \% 7$$
$$= [6 + 1 * (1 + 692 \% 5)] \% 7$$
$$= 9 \% 7$$
$$= 2$$
Now, as 2 is an empty slot, so we can insert 692 into 2nd slot.

Slot

0	
1	43
2	692
3	
4	
5	72
6	27

Step 05

The next key is 72 which is mapped to slot 2 ($72\%7=2$), but location 2 is already occupied. Using double hashing,
$$h_{new} = [h_1(72) + i * (h_2(72))] \% 7$$
$$= [2 + 1 * (1 + 72 \% 5)] \% 7$$
$$= 5 \% 7$$
$$= 5,$$
Now, as 5 is an empty slot, so we can insert 72 into 5th slot.